
Beyond Output Filtering: Externally Auditable Erasure for Inference-Time RAG Memory

Sean Culatana
Atlassian
Mountain View, CA
sculatana@atlassian.com

Kang Li
Atlassian
Bellevue, WA
kli3@atlassian.com

Abstract

Retrieval-augmented systems increasingly rely on vector indexes that may retain deleted items in their search graph. Existing deletion interfaces can prevent deleted identifiers from appearing in returned results while still computing distances to their embeddings during graph traversal. We formalize this distinction as *output safety* versus *traversal safety*, and introduce TSD-AUDIT, a framework for auditing and enforcing traversal-safe deletion in graph-based approximate nearest-neighbor retrieval. On Faiss IndexHNSWFlat, native filtering leaves the number of distance computations unchanged relative to unfiltered search; at a 70% deletion rate, trace-faithful replay detects deleted-vector scoring in all 100 audited queries. Code inspection of hnsplib’s `mark_deleted` path reveals the same scoring-before-liveness pattern. TSD-AUDIT enforces an *alive-before-scoring* invariant, repairs connectivity using only live candidates, and emits per-query scored-trace certificates that an independent verifier can check against the deletion snapshot. Under region-targeted deletion, TSD-AUDIT improves Recall@10 over native filtering by 4.3–42.2 percentage points across deletion fractions from 0.5 to 0.9, while remaining comparable under random deletion. These results show that output-only deletion audits can miss process-level exposure: auditing deletion in vector retrieval requires accounting for the vectors scored during search, not only the identifiers returned.

1 Introduction

Privacy research has historically anchored on the training phase, relying predominantly on differentially private training such as DP-SGD. But state-of-the-art applications increasingly pivot to inference-time, non-finetuning paradigms: autonomous agents, in-context learning, and retrieval-augmented generation. In these deployments a large share of the sensitive data an application handles never enters model weights at all—it sits in a *retrieval memory* (a vector index over user documents, conversations, or tool traces) that is read at inference time. This relocates a core privacy obligation from training to deployment: when a user exercises a right to erasure, or an operator must expunge disallowed or poisoned content, the memory must delete the entry *and* the system must be able to *prove* the deletion.

We study this obligation for the component most inference-time memories are built on: an approximate-nearest-neighbor (ANN) vector index. Production vector stores implement deletion as *soft-delete*: a tombstone filters deleted ids from the returned top- k , but the graph search still visits and scores the deleted embeddings (Figure 1). We argue this makes the standard, output-level compliance

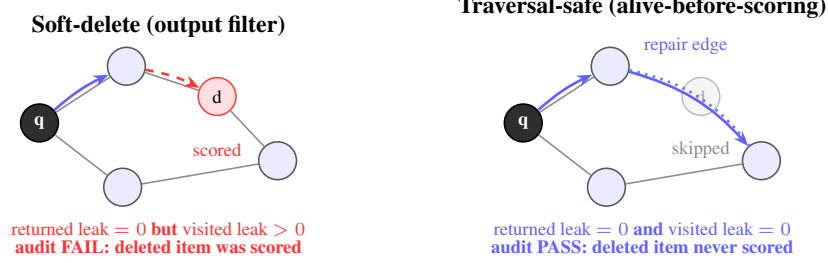


Figure 1: **Returned vs. visited leakage.** A deleted memory entry d is filtered from the returned top- k under both regimes, so an output-level audit sees zero leakage in each. *Left:* soft-delete still *visits and scores* d (dashed red), so the retrieval process reads erased data on nearly every query—invisible to an output audit. *Right:* the alive-before-scoring invariant never admits d to the candidate pool and an offline repair edge restores connectivity, driving visited leakage to zero and yielding an externally checkable certificate.

audit blind by construction: it observes returned results, where leakage is zero, and never observes the retrieval process, where a deleted embedding is read on nearly every query. An assistant that “forgot” a user’s revoked record at the output while still reading it internally on every query has not really forgotten it—and no output-level audit can tell the difference. We therefore elevate deletion from a returned-result constraint to an *auditable retrieval-time guarantee*, and—unlike prior soft-delete analyses that use a reimplementation—establish the leak on the *unmodified faiss* engine and emit a certificate an external party can check.

Our contribution is deliberately scoped as *audit-not-algorithm*. The enforcement mechanism (an alive predicate checked before scoring) is the label-filtered traversal of predicate-aware ANN (Patel et al., 2024; Gollapudi et al., 2023); the recall-collapse-and-repair phenomenon is known churn behavior (Singh et al., 2021; Xiao et al., 2024); and at-rest recoverability of soft-deleted vectors is studied by Ghost Vectors (Chakrabortii et al., 2026). What none of these own is a certifiable, externally verifiable guarantee that the retrieval *process* never scored a deleted item—an emerging, inference-time privacy problem. We contribute: (1) the output-safe vs. traversal-safe distinction as measurable leakage; (2) the alive-before-scoring invariant and a traversal-safe repair; (3) an externally verifiable certificate of erasure; and (4) an *external-validity audit on unmodified faiss* plus an *honest baseline reconciliation* against faiss’s real soft-delete.

2 Threat model and leakage

We assume an *honest-but-buggy* operator running an evolving inference-time pipeline (ANN retrieval feeding a reranker/LLM, with caches and logs). “Compliant” means no query-time operation reads or computes on a deleted item; “auditable” means this is checkable from emitted evidence without trusting the code path. We do *not* target a malicious operator forging certificates, nor parameter-level unlearning of model weights (Bourtoule et al., 2021). Let $\text{Visited}(q, t)$ be every node on which the search computes a query–node distance. **Returned leakage** is the rate at which a deleted id appears in the top- k ; **visited leakage** is the rate at which any deleted node is scored. A system is **traversal-safe** if both are zero. We stress the scope of the exposure: it is observable to a *process-level* party (an auditor, or the operator’s own telemetry). A purely client-observable adversary restricted to returned ids/scores is at chance—returned leakage is 0—so the exposure is a process/compliance property of erasure, not a client-exploitable side channel. This is precisely the regime a deployment-time privacy audit must cover and an output test cannot.

3 Method

Alive-before-scoring invariant. The memory maintains an alive bitset consulted at three points, all *before* a distance is computed: entry points are drawn from live nodes; deleted neighbors are dropped from the frontier before scoring; and a pop-time re-check skips nodes deleted after enqueue. Because deleted nodes never enter the candidate pool, $\text{Visited}(q, t) \cap D_t = \emptyset$ holds for every query

by construction. **Traversal-safe repair.** Pruning dead edges does not restore the connectivity deleted nodes provided; at high deletion the live subgraph fragments. An offline repair reconnects live nodes through paths that ran through deleted ones, scoring distances only among *live* candidates, so it preserves the guarantee. **Certificate.** At each checkpoint we emit the deleted-set hash, hashes of sampled expansion traces, and the visited $\cap D_t$ verdict; a released verifier recomputes the hashes and asserts no-intersection, turning traversal-safe deletion into an externally checkable artifact.

4 External validity on the unmodified faiss engine

To rule out that the leak is an artifact of our own beam search, we audit the *native* faiss 1.11.0 HNSW search path (IndexHNSWFlat, $M=16$, efC=200, efS=128) over 114,516 bge vectors, using faiss’s own hnsw_stats.ndis counter—no custom code in the loop—at 70% deletion (3 seeds), across three deployment conditions: (A) *soft-delete* (plain search, filter output); (B) *native selector* (SearchParametersHNSW with IDSelectorBatch(alive)); (C) *clean rebuild* on the live set.

The engine’s own counter shows the built-in selector is output-safe, not traversal-safe: $\text{ndis}(A)=\text{ndis}(B)=1,535,152$ *exactly*, every seed and both deletion modes (ratio 1.000). faiss’s IDSelectorBatch does not prune before the distance kernel; deleted embeddings are still scored, only removed from output. Only rebuilding (C) stops the engine scoring them (ndis $1.05\times$ random, $1.38\times$ targeted). Returned leakage is 0 in all conditions. We then emit the certificate from a replay whose fidelity to the native engine we validate directly (top-10 agreement 1.000; replayed distance count $0.98\times$ faiss’s own ndis) and run the *released* verifier over 100 sampled queries: the soft-delete certificate **fails** (a deleted id in the scored set of every sampled query), while traversal-safe and clean-rebuild **pass** (0 hash/count mismatches; Table 1).

Table 1: External-validity audit on unmodified faiss (70% deletion, 3 seeds). The native selector leaves distance work unchanged; the released verifier fails soft-delete and passes the traversal-safe and rebuild certificates. Take-away: output filtering does not stop the memory from reading deleted entries.

Condition	ndis (targeted)	returned leak	verifier
A. soft-delete (output filter)	1,535,152	0	FAIL
B. native IDSelectorBatch	1,535,152	0	FAIL
C. clean rebuild (live only)	2,124,533	0	PASS
Traversal-safe (ours)	—	0	PASS

5 Honest recall reconciliation

A common but *weak* soft-delete baseline oversamples a fixed $k'>k$ then filters; under targeted deletion this leaves few live results and collapses to near-zero recall. faiss’s real soft-delete (native selector, fill- k) is far stronger. We therefore reconcile our recall claims against faiss’s real soft-delete on the same graph (Figure 2). Two honest findings. **(i)** Under *random* deletion there is no quality advantage (all methods ≈ 0.99). **(ii)** Under *targeted/region* deletion—the realistic case for erasing a user, a topic, or a compromised source—traversal-safe+repair beats faiss’s real soft-delete by a margin that grows with the deletion fraction (+4.3/+16.2/+42.2 points at $f=0.5/0.7/0.9$), far smaller than the gap against the weak baseline. A stop-the-world clean rebuild attains the highest raw recall (≥ 0.97) at every fraction; traversal-safe+repair is therefore best read as an *online, availability-preserving, certificate-clean* recovery that beats real soft-delete, not as a replacement for a rebuild on raw quality.

6 Related work and scope

Graph ANN (Malkov and Yashunin, 2016; Subramanya et al., 2019) in libraries such as faiss (Douze et al., 2024) and dynamic benchmarks (Simhadri et al., 2024) make dynamic retrieval practically important; recent work studies deletion algorithmically (Xu et al., 2025) and as evaluation (Yamashita et al., 2025). We concede the alive-filter mechanism to predicate-filtered ANN (Patel et al., 2024; Gollapudi et al., 2023), the recall-collapse-and-repair phenomenon to the churn literature (Singh et al., 2021; Xiao et al., 2024), and at-rest recoverability to Ghost Vectors (Chakrabortii et al., 2026).

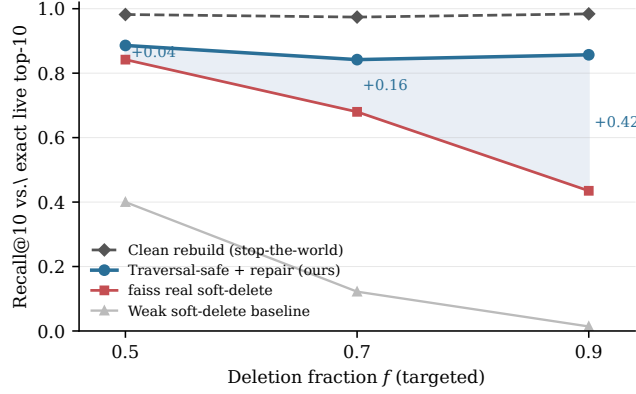


Figure 2: **Recall@10 vs. exact live top-10 under targeted deletion** (mean of 3 seeds), against *faiss*’s *real* soft-delete. The shaded band is the honest gap (traversal-safe+repair – real soft-delete): +0.043/+0.162/+0.422 at $f=0.5/0.7/0.9$. Auditable erasure need not cost retrieval quality under targeted deletion; a stop-the-world rebuild leads on raw recall at rebuild cost.

Verifiable-ANN certifies result *correctness* (Wang et al., 2024) and factual-residual audits operate at the model level (Raeesi and Roed, 2026); neither certifies the retrieval process of an inference-time memory. Our residual, defensible contribution is the traversal-exposure *audit property* for erasure and its externally verifiable certificate.

7 Limitations

The exposure is a process/compliance property, not a client-exploitable channel (returned leakage is 0). Our quality benefit is scoped to targeted/region deletion and grows with f ; under random deletion it vanishes, and a stop-the-world rebuild dominates raw recall at rebuild cost. Certificates are scoped to an honest-but-buggy operator; a malicious operator forging traces requires a trusted execution environment or transparency log, which we leave to future work. We study the RAG memory layer specifically; auditable erasure for parametric or in-context memory is complementary and out of scope.

References

- Lucas Bourtole, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine unlearning. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.
- Chandranil Chakrabortii, Jackeline García Alvarado, Sitora Abdulofizova, and Shivanshu Dwivedi. Ghost vectors: Soft-deleted embeddings remain reconstructible in HNSW vector databases. *arXiv preprint arXiv:2606.18497*, 2026.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The Faiss library. *arXiv preprint arXiv:2401.08281*, 2024.
- Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapneel Raje, Zhu Lin, Yiyong Zhang, Neelam Mahapatro, Zohar Karnin, Amit Srinivasan, and Harsha Vardhan Simhadri. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *The Web Conference (WWW)*, 2023.
- Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *arXiv preprint arXiv:1603.09320*, 2016.
- Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. ACORN: Performant and predicate-agnostic search over vector embeddings and structured data. *Proc. ACM Manag. Data (SIGMOD)*, 2(3), 2024.

- Arya Raeesi and Hanna Roed. Auditing forgetting in limited memory language models. *arXiv preprint arXiv:2607.00605*, 2026.
- Harsha Vardhan Simhadri, Martin Aumüller, Matthijs Douze, Dmitry Baranchuk, Amir Ingber, Edo Liberty, George Williams, Ben Landrum, Magdalen Dobson Manohar, and others. Results of the Big ANN: NeurIPS’23 competition. *arXiv preprint arXiv:2409.17424*, 2024.
- Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search. *arXiv preprint arXiv:2105.09613*, 2021.
- Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. DiskANN: Fast accurate billion-point nearest neighbor search on a single node. In *NeurIPS*, 2019.
- Chenzhao Wang, Jilian Zhang, Xuyang Liu, Kaimin Wei, and Bingwen Feng. Verifiable graph-based approximate nearest neighbor search. In *Advanced Data Mining and Applications (ADMA)*, Springer, 2024.
- Wentao Xiao, Yueyang Zhan, Rui Xi, Mengshu Hou, and Jianming Liao. Enhancing HNSW index for real-time updates: Addressing unreachable points and performance degradation. *arXiv preprint arXiv:2407.07871*, 2024.
- Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. In-place updates of a graph index for streaming approximate nearest neighbor search. *arXiv preprint arXiv:2502.13826*, 2025.
- Tomohiro Yamashita, Daichi Amagata, and Yusuke Matsui. How should we evaluate data deletion in graph-based ANN indexes? *arXiv preprint arXiv:2512.06200*, 2025.